



河南大學
HENAN UNIVERSITY



Web编程基础

(2020-2021-2)

任课教师: 丁爽



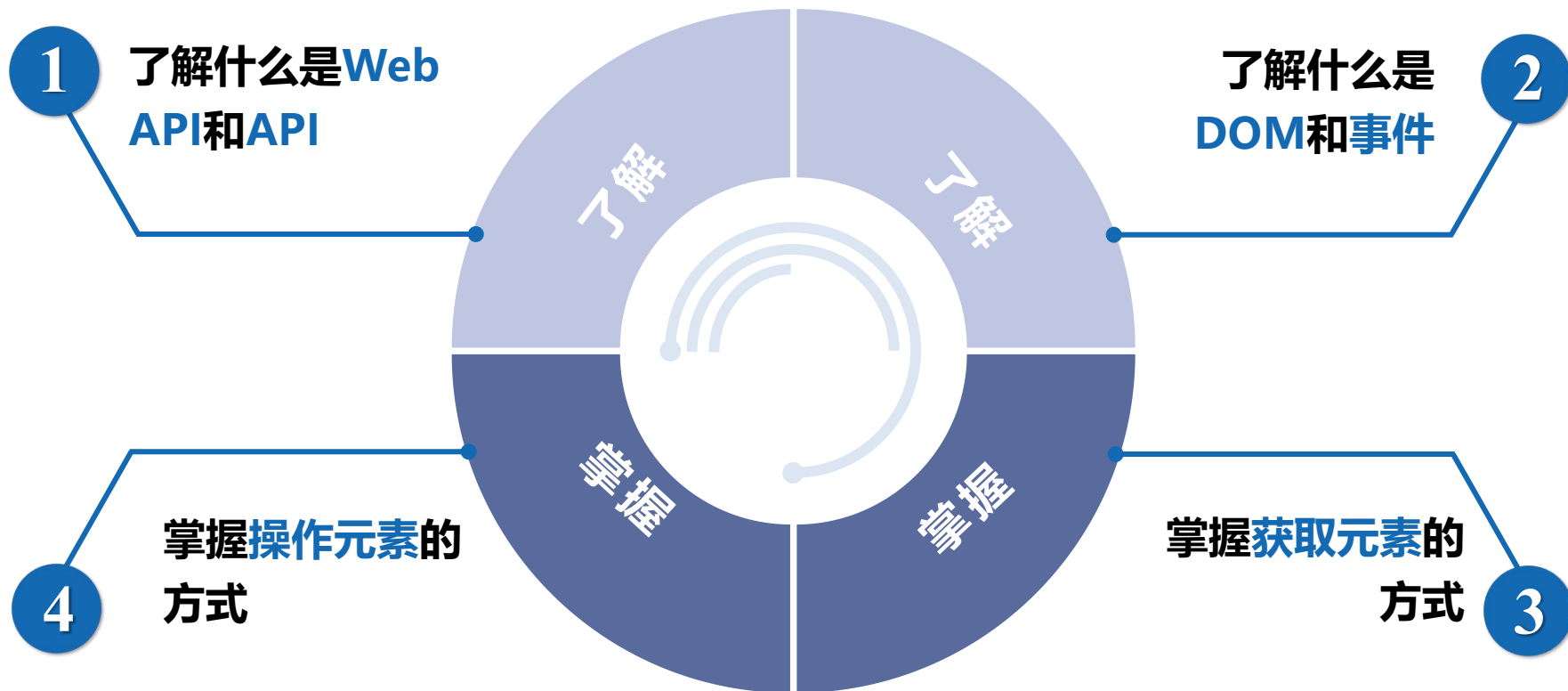
内容回顾

- 函数：声明、使用、参数和返回值的设置、**arguments**;
- 函数表达式，回调函数，递归调用
- 作用域：全局变量和局部变量，作用域链
- 对象：属性、方法，访问方法
- 创建对象：字面量、**new Object**、构造函数
- 内置对象：数学对象**Math**、日期对象**Date**、数组对象**Array**、字符串对象**String**



第6章 DOM (上)

学习目标



目录



6.1

Web API简介

👉 [点击查看本节相关知识点](#)

6.2

DOM简介

👉 [点击查看本节相关知识点](#)

6.3

获取元素

👉 [点击查看本节相关知识点](#)

6.4

事件基础

👉 [点击查看本节相关知识点](#)

目录



操作元素

👉 [点击查看本节相关知识点](#)



6.1 Web API简介

1

初识Web API

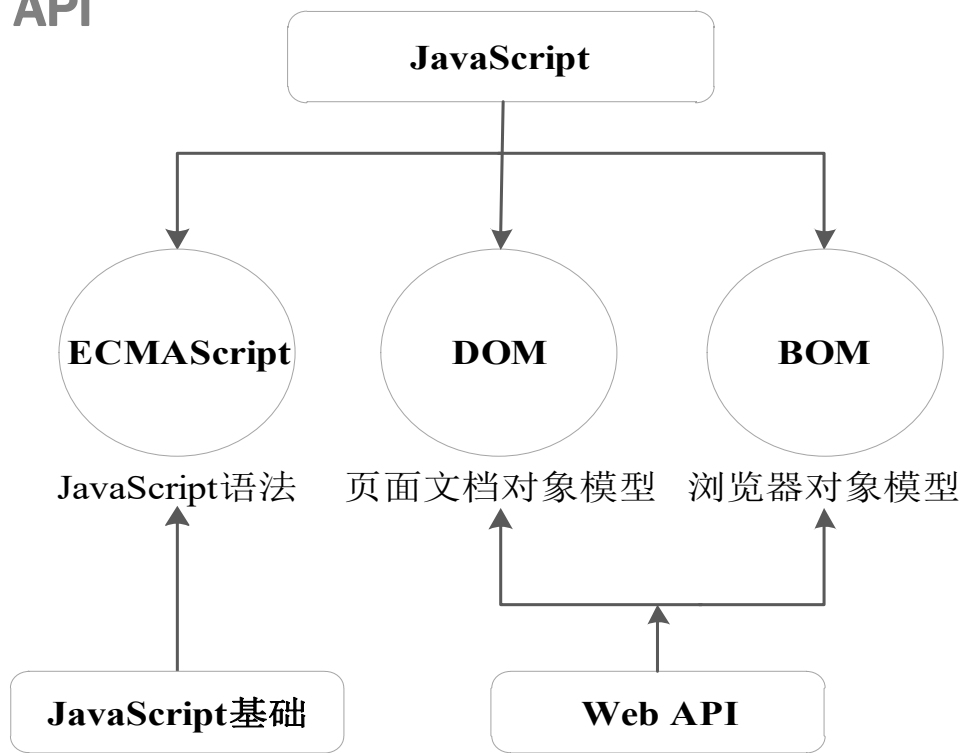
Web API是浏览器提供的一套操作浏览器功能和页面元素的接口。JavaScript语言由3部分组成，分别是ECMAScript、BOM和DOM，其中ECMAScript是JavaScript语言的核心，而Web API包括BOM和DOM两部分。



6.1 Web API简介

1

初识Web API



JavaScript的组成部分

6.1 Web API简介



2 Web API与API的关系

API: Application Programming Interface, 应用程序编程接口, 是一些预先定义的函数, 这些函数是由某个软件开放给开发人员使用的, 帮助开发者实现某种功能, 开发人员无须访问源码、无须理解其内部工作机制细节, 只需知道如何使用即可。例如, 调用手机的摄像头拍摄画面。

开发人员查阅API文档, 了解API如何使用



6.1 Web API简介

2 Web API与API的关系

Web API: 主要针对浏览器的**API**，在JavaScript语言中被封装成了对象，通过调用对象的属性和方法就可以使用**Web API**。例如，`console`对象、`document`对象、`window`对象。

示例代码

```
document.title = '设置新标题';           // 设置页面标题
console.log(document.title);             // 获取页面标题
document.write('<h1>网页内容</h1>');      // 将字符串写入页面
```



6.2 DOM简介

1

什么是DOM

DOM: Document Object Model, 文档对象模型, 是W3C组织推荐的**处理可扩展标记语言（HTML或者XML）**的标准编程接口。

W3C定义了一系列的DOM接口, 利用DOM可完成对HTML文档内所有元素的**获取**、**访问**、**标签属性**和**样式的设置**等操作。在实际开发中, 诸如改变盒子的大小、标签栏的切换、购物车功能等带有交互效果的页面, 都离不开DOM。

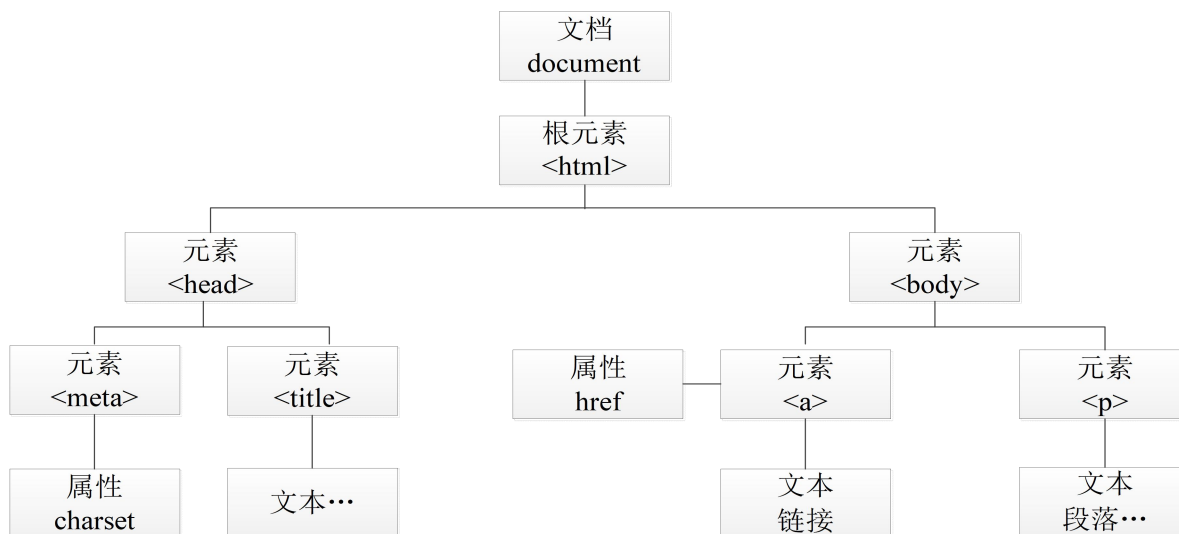


6.2 DOM简介

2

DOM树

DOM中将HTML文档视为树结构，被称之为文档树模型，把文档映射成树形结构，通过节点对象对其处理，处理的结果可以加入到当前的页面。



DOM树



6.2 DOM简介

2

DOM树

接下来针对DOM中各节点的专有名词解释如下：

- ❑ 文档（document）：可以把一个页面当成一个文档
- ❑ 元素（element）：页面中的所有标签都是元素
- ❑ 节点（node）：网页中的所有内容，在文档树中都是节点（如：元素节点、属性节点、文本节点、注释节点等），在DOM中会把所有的节点都看作是对象，这些对象拥有自己的属性和方法



6.3 获取元素

1

根据id获取元素

[document.getElementById\('id'\)](#)方法，是由document对象提供的用于查找元素的方法，该方法返回的是拥有指定id的元素，如果没有找到指定id的元素则返回null，如果存在多个指定id的元素则返回undefined。

示例代码

```
<body>
  <div id="box">你好</div>
  <script>
    var Obox = document.getElementById('box');
    console.log(Obox);      // 结果为: <div id="box">你好</div>
    console.log(typeof Obox); // 结果为: object
    console.dir(Obox);      // 结果为: div#box
  </script>
</body>
```



6.3 获取元素

2

根据标签获取元素

根据标签名获取元素的两种方式：可以通过[document对象](#)获取元素和通过[element对象](#)获取元素。

语法结构

```
document.getElementsByTagName('标签名');  
element.getElementsByTagName('标签名');
```

由于相同标签名的元素可能有多个，上述方法返回的不是单个元素对象，而是一个[集合](#)。这个集合是一个类数组对象，或称为[伪数组](#)，它可以像数组一样用索引来访问元素，但不能使用[push\(\)](#)等方法，使用[Array.isArray\(\)](#)也可以证明它不是一个数组。



6.3 获取元素

2

根据标签获取元素

```
<ul>
  <li>苹果</li>
  <li>香蕉</li>
  <li>西瓜</li>
  <li>樱桃</li>
</ul>
<ol id="ol">
  <li>绿色</li>
  <li>蓝色</li>
  <li>白色</li>
  <li>红色</li>
</ol>
```

```
var lis = document.getElementsByTagName('li');
// 结果为: HTMLCollection(8) [li, li, li, li, li, li, li, li]
console.log(lis);    lis的构造函数式HTMLCollection
// 查看集合中的索引为0的元素, 结果为: <li>苹果</li>
console.log(lis[0]);
// 遍历集合中的所有元素
for (var i = 0; i < lis.length; i++) {
  console.log(lis[i]);
}
// 通过元素对象获取元素
var ol = document.getElementById('ol');
// 结果为: HTMLCollection(4) [li, li, li, li]
console.log(ol.getElementsByTagName('li'));
```

`element.getElementsByTagName()`, `element`必须是单个对象元素



6.3 获取元素

2

根据标签获取元素

注意

`getElementsByTagName()`方法获取到的集合是动态集合，也就是说，当页面增加了标签，这个集合中也会自动增加元素。



6.3 获取元素

3 根据name获取元素

[document.getElementsByName\(\)](#)方法，是通过name属性来获取元素，一般用于获取表单元素。name属性的值不要求必须是唯一的，多个元素也可以有同样的名字，如表单中的单选框和复选框。

语法结构

```
document.getElementsByName('name');
```



6.3 获取元素

3 根据name获取元素

案例需求: 在复选框中选择最喜欢的水果（多选）。

示例代码

```
<body>
  <p>请选择你最喜欢的水果（多选） </p>
  <label> <input type="checkbox" name="fruit" value="苹果">苹果</label>
  <label> <input type="checkbox" name="fruit" value="香蕉">香蕉</label>
  <label> <input type="checkbox" name="fruit" value="西瓜">西瓜</label>
  <script>
    var fruits = document.getElementsByName('fruit'); // 返回对象集合
    fruits[0].checked = true; // 将fruits中的第1个元素的checked属性值设置为true
  </script>
</body>
```



6.3 获取元素

3 根据name获取元素



根据name获取元素



6.3 获取元素

4

HTML5新增的获取方式

HTML5中为document对象新增了[querySelector\(\)](#)、[querySelectorAll\(\)](#)和[getElementsByName\(\)](#)方法。注意：这三种方式在使用时需要考虑**浏览器的兼容性问题**。



6.3 获取元素

4

HTML5新增的获取方式

① 根据类名获取

[document.getElementsByClassName\(\)](#)方法，用于通过类名来获得某些元素集合。



6.3 获取元素

4

HTML5新增的获取方式

示例代码

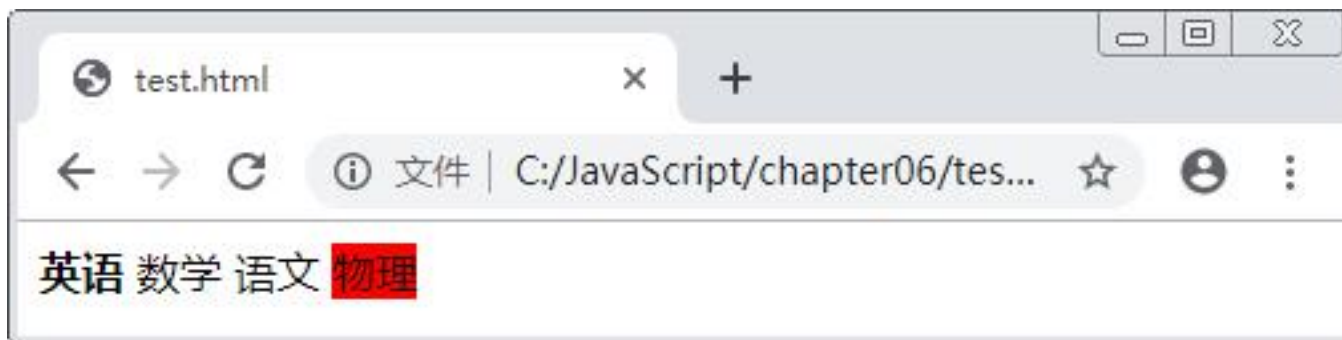
```
<body>
  <span class="one">英语</span> <span class="two">数学</span>
  <span class="one">语文</span> <span class="two">物理</span>
  <script>
    var Ospan1 = document.getElementsByClassName('one');
    var Ospan2 = document.getElementsByClassName('two');
    Ospan1[0].style.fontWeight = 'bold';
    Ospan2[1].style.background = 'red';
  </script>
</body>
```



6.3 获取元素

4

HTML5新增的获取方式



通过类名获取元素



6.3 获取元素

4

HTML5新增的获取方式

② querySelector()和querySelectorAll()

querySelector()方法用于返回指定选择器的第一个元素对象。

querySelectorAll()方法返回指定选择器的所有元素对象集合。

结构代码

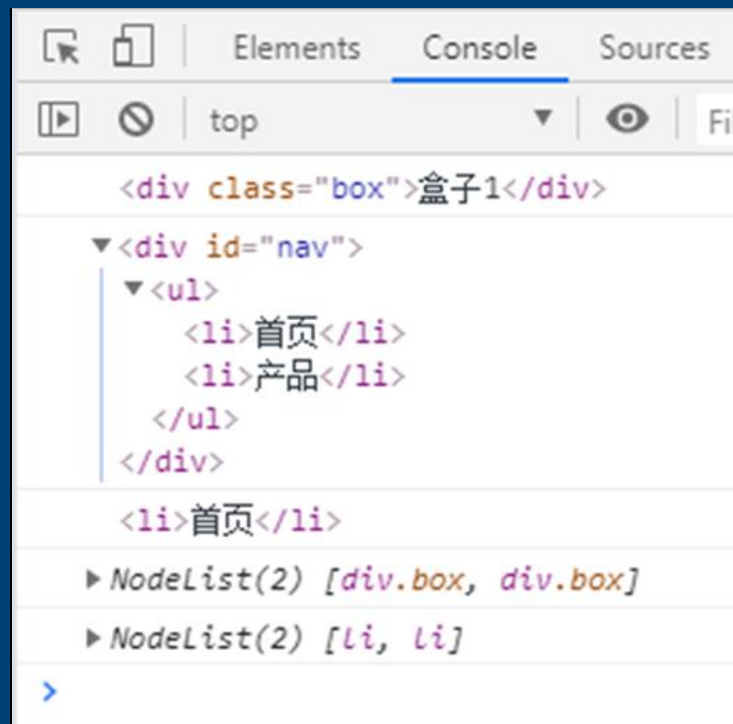
```
<body>
  <div class="box">盒子1 </div>
  <div class="box">盒子2 </div>
  <div id="nav">
```

JS代码

```
<script>
  var firstBox = document.querySelector('.box');
  console.log(firstBox); // 获取class为box的第1个div
  var nav = document.querySelector('#nav');
  console.log(nav);      // 获取id为nav的第1个div
  var li = document.querySelector('li');
  console.log(li);       // 获取匹配到的第一个li
  var allBox = document.querySelectorAll('.box');
  console.log(allBox);   // 获取class为box的所有div
  var lis = document.querySelectorAll('li');
  console.log(lis);      // 获取匹配到的所有li
</script>
```

结构代码

```
<ul>
  <li>首页</li>
  <li>产品</li>
</ul>
</div>
</body>
```



通过CSS选择器获取元素



6.3 获取元素

5

document对象的属性

document对象提供了一些属性，可用于获取文档中的元素。例如，获取所有表单标签、图片标签等，document对象的常用属性如下表：

属性	作用
document.body	返回文档的body元素
document.title	返回文档的title元素
document.documentElement	返回文档的html元素
document.forms	返回对文档中所有Form对象引用
document.images	返回对文档中所有Image对象引用



6.3 获取元素

5

document对象的属性

案例演示: 以获取body元素和html元素为例进行代码演示。

示例代码

```
<body>
<script>
  var bodyEle = document.body;
  console.dir(bodyEle);
  var htmlEle = document.documentElement;
  console.log(htmlEle);
</script>
</body>
```

